

1.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

y_prob = model.predict_proba(X_test)[:,1]
print("ROC-AUC Score :", roc_auc_score(y_test, y_prob))
```

Accuracy : 0.9649122807017544

Confusion Matrix:

[[40 3]

[1 70]]

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

ROC-AUC Score : 0.9952505732066819

2.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)

df = pd.read_csv("/content/diabetes.csv")
df.columns = ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
              "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"]

cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[cols] = df[cols].replace(0, df[cols].median())

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

y_prob = model.predict_proba(X_test)[:,1]
print("ROC-AUC :", roc_auc_score(y_test, y_prob))

coef = pd.DataFrame(model.coef_[0], index=X.columns, columns=["Coefficient"])
print(coef)

```

```

Accuracy : 0.7662337662337663
Confusion Matrix:
[[82 17]
 [19 36]]

```

	precision	recall	f1-score	support
0	0.81	0.83	0.82	99
1	0.68	0.65	0.67	55
accuracy			0.77	154
macro avg	0.75	0.74	0.74	154
weighted avg	0.76	0.77	0.77	154

```

ROC-AUC : 0.8196510560146923

```

	Coefficient
Pregnancies	0.218415
Glucose	1.138214
BloodPressure	-0.165564
SkinThickness	0.049443
Insulin	-0.224917
BMI	0.718367
DiabetesPedigreeFunction	0.216745
Age	0.378817

3

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)

df = pd.read_csv("/content/titanic.csv")

df["Age"] = df["Age"].fillna(df["Age"].median())
df["Fare"] = df["Fare"].fillna(df["Fare"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

df = df.drop(["Name", "Ticket", "Cabin"], axis=1)

le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])
df["Embarked"] = le.fit_transform(df["Embarked"])

X = df.drop("Survived", axis=1)
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=4,
    random_state=42,
    eval_metric="logloss"
)

model.fit(X_train, y_train)

```

```

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))

print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

print("\nFeature Importance")
print(importance.sort_values(by="Importance", ascending=False))

```

Accuracy: 0.8100558659217877

Confusion Matrix

```
[[92 13]
 [21 53]]
```

Classification Report

	precision	recall	f1-score	support
0	0.81	0.88	0.84	105
1	0.80	0.72	0.76	74
accuracy			0.81	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

ROC-AUC Score: 0.8747104247104248

Feature Importance

	Feature	Importance
2	Sex	0.584156
1	Pclass	0.177682
4	SibSp	0.058212
6	Fare	0.047285
7	Embarked	0.042100
3	Age	0.039796
0	PassengerId	0.031227
5	Parch	0.019542

4

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

df = pd.read_csv("/content/diabetes_2(1).csv")

df.columns = ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
              "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"]

cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[cols] = df[cols].replace(0, df[cols].median())

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model1 = DecisionTreeClassifier(random_state=42)
model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

print("Full Tree Accuracy :", accuracy_score(y_test, pred1))
print(confusion_matrix(y_test, pred1))

```

```

print(classification_report(y_test, pred1))

model2 = DecisionTreeClassifier(max_depth=3, random_state=42)
model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

print("Depth=3 Accuracy :", accuracy_score(y_test, pred2))
print(confusion_matrix(y_test, pred2))
print(classification_report(y_test, pred2))

importance = pd.DataFrame(
    model2.feature_importances_,
    index=X.columns,
    columns=["Importance"]
)

print(importance.sort_values("Importance", ascending=False))

```

Full Tree Accuracy : 0.7272727272727273

[[74 25]

[17 38]]

	precision	recall	f1-score	support
0	0.81	0.75	0.78	99
1	0.60	0.69	0.64	55
accuracy			0.73	154
macro avg	0.71	0.72	0.71	154
weighted avg	0.74	0.73	0.73	154

Depth=3 Accuracy : 0.7597402597402597

[[83 16]

[21 34]]

	precision	recall	f1-score	support
0	0.80	0.84	0.82	99
1	0.68	0.62	0.65	55
accuracy			0.76	154
macro avg	0.74	0.73	0.73	154
weighted avg	0.76	0.76	0.76	154

	Importance
Glucose	0.596625
BMI	0.263305
Age	0.140070
Pregnancies	0.000000
SkinThickness	0.000000
BloodPressure	0.000000
Insulin	0.000000
DiabetesPedigreeFunction	0.000000